勉強日記

日付

2026年04月16日

学習内容

[Amazon SageMaker: 推論レイテンシーのトラブルシューティングとModelSetupTime]

- **概要**: 本番環境で推論レイテンシーがテスト時のベースラインより増加した場合、その原因がモデルの起動時間にあるという仮説を検証するために、CloudWatchでSageMaker名前空間の `ModelSetupTime` メトリクスを監視する。
- **試験ポイント**:
  - 「Model Monitor」はモデルの品質（データドリフトや精度低下など）を監視するものであり、起動時間などのインフラレイテンシーの特定には使えないという「引っかけ」に注意する。
  - `ModelLoadingWaitTime` はモデルの読み込み待機時間のみであり、起動時間全体を示さない。モデルの起動全体に起因するレイテンシー増加を確認するには、必ず `ModelSetupTime` を選択する。
- **実務での使い方**: マルチモデルエンドポイント（MME）やサーバーレス推論など、リクエストに応じてインスタンスにモデルがロードされる構成において発生するコールドスタートやボトルネックを分析する際に重要。この指標がボトルネックとなっている場合は、モデルの軽量化や推論環境のサイジング・プロビジョニング戦略を見直す。
- **副業での需要**: ★★★☆☆
  - 運用保守やパフォーマンス改善の案件で求められる知識。「推論APIのレスポンスが遅い」という課題に対して、勘に頼らずCloudWatchメトリクスから論理的に原因を特定し、インフラチューニングを提案・実行するスキルは、運用フェーズの副業（時給構成の保守案件など）で評価される。

[Amazon SageMaker: 外部モデルの持ち込み（BYOM）とModel Registry]

- **概要**: ローカル環境（ラップトップ等）で開発されたscikit-learnやTensorFlowのモデルをSageMakerで実行するには、単にModel Registryにインポートするだけでは動作しない。適切なSageMakerコンテナやフレームワーク設定を用いてパッケージング（構築）し直す必要がある。
- **試験ポイント**:
  - 「Model Registry」はあくまでモデルのパッキングやバージョン、メタデータを「管理」するためのサービスであり、ローカルのモデルを直接インポートして自動で実行可能にする「マジック」ではない、という点が出題の引っかけとなる。

    - 外部モデルの持ち込み（BYOM: Bring Your Own Model）の際は、AWS提供の構築済みコンテナイメージ（あるいはカスタムコンテナ）を使用し、指定の形式（`model.tar.gz`）と推論スクリプトを用意してSageMaker環境用にエンドポイントを再構築する必要がある。
- **実務での使い方**: データサイエンティストが手元で検証したモデルを本番環境へ移行する「ローカル/オンプレ → クラウド」のシナリオで必須の知識。推論用スクリプト（`inference.py`）と`requirements.txt`を用意し、AWS公式の推論コンテナにモデルアーティファクトを連携させて堅牢なAPI化を行う。
- **副業での需要**: ★★★★☆
    - 「手元で作ったモデルをWebサービスとして公開（API化）したい」という案件で需要が高い。既存のモデル資産を活かしつつ、SageMakerのマネージドなエンドポイントとして適切にコンテナ連携・デプロイできるスキルを持つと、システム構築案件を高単価で獲得しやすい。

[Amazon SageMaker Ground Truth: プライベートワーカーフォース vs Mechanical Turk]

- **概要**: 画像・テキストのデータラベリングにおいて、機密性の高いデータには組織内従業員限定である「プライベートワーカーフォース」を使用する。個人情報を含まない一般的なタスクにはクラウドソーシングの「Amazon Mechanical Turk（パブリック）」を利用できる。
- **試験ポイント**:
    - 「機密データ（PIIなど）を扱う」「社内の従業員のみで処理する」という要件がある場合、Mechanical Turkを選択するのはセキュリティリスクとなるため明確な誤り（引っかけ）。必ず「プライベートワーカーフォース」を選択する。
    - プライベートワーカーフォースに「レビュープロセス（多重チェック）」を組み合わせることで、データの機密性を担保しつつ高品質なラベリングを実現できるという構成がよく出題される。
- **実務での使い方**: 医療データ、財務データ、顧客の顔写真など、社外秘データのラベリング基盤を構築する際に不可欠な機能。Amazon Cognitoなどの認証機能と連携させ、社内の特定部門の社員だけがセキュアにアノテーションUIへアクセスできるようにアクセス制御を行う。
- **副業での需要**: ★★☆☆☆
    - データラベリング業務自体は企業内で完結させるか専門のBPO企業に委託されることが多いため、副業エンジニアがワーカー管理システムを作る案件は多くはない。ただし、MLシステム構築において「機密データの取り扱いに関するコンプライアンス要件（セキュリティ設計）」を含めてアドバイス・設計できると、単価の高いアーキテクト的な立ち位置で重宝される。

[Amazon Comprehend: エンティティ抽出とIDP（Intelligent Document Processing）機能]

- **概要**: テキストからエンティティ（人物、組織、場所、日付など）を自動抽出するフルマネージドな事前学習済みNLPサービス。IDP機能により、事前にテキスト化レイヤーを挟むことなく、PDFやWord、画像ファイル等の半構造化ドキュメントを「直接」処理し、結果をS3へ出力できるようになった。

- **試験ポイント**:
  - エンティティ抽出要件に対して「独自のモデルを構築する（SageMaker等）」という選択肢は、要件が特化していない限り開発・運用コストの観点から不正解（引っかけ）になることが多い。正解は「ComprehendのAPIを呼び出す」。
  - PDFドキュメント等から処理時間を最小限にして事前学習済みモデルでエンティティ抽出する問題では、「非同期分析（最大50MB、100ページ）で直接S3に結果を保存する仕組み」を選ぶのがベストプラクティス。
- **実務での使い方**: 契約書、請求書、コールセンターの応対履歴、アンケート結果などの大量の非構造化・半構造化ドキュメント（PDF等）をS3にアップロードし、EventBridge + LambdaからComprehendの非同期APIを叩いて、抽出結果を別のS3バケットにLake House的に蓄積するパイプラインで利用する。自力でOCR + NLPモデルを構築する手間を劇的に削減できる。
- **副業での需要**: ★★★☆☆
  - 「社内に眠っている大量のPDFやドキュメントを分析可能なデータに変換したい」という企業のDX系案件で非常に強力な武器になる。機械学習の深い専門知識（モデル開発）がなくても、AWSのAPI群を繋ぎ合わせる（S3 + Step Functions/Lambda + Comprehend/Textract）だけで付加価値の高いPoCシステムを迅速に構築・納品できるため、スポット案件でも稼ぎやすい。

[AWS KMS & Encryption CLI: データの暗号化と細やかなアクセス制御]

- **概要**: AWS KMS（Key Management Service）は、暗号化キーの作成・管理を行うマネージドサービス。ファイルの暗号化/復号化には「AWS Encryption CLI」を利用し、キーへのアクセス権限をIAMポリシーとKMSキーポリシーの両方で厳密に制御することで、暗号化とアプリケーション単位のアクセス制御課題を同時に解決できる。
- **試験ポイント**:
  - データ暗号化要件に加えて「特定のアプリケーションやRoleだけがキーにアクセスできるようにしたい」というアクセス制御の要件が組み合わさった問題では、S3のデフォルト暗号化ではなく「カスタマー管理のKMSキー（CMK）」と「IAM/キーポリシーによる権限制御」の組み合わせが正解となる。
  - ローカルファイルやスクリプトでの暗号化処理を実行するためのツールとして「AWS Encryption CLI」を利用する設計パターンが出題される。
- **実務での使い方**: 機密性の高いML学習用データ（PIIや財務データなど）を扱う場合、S3にアップロードする前にオンプレミスやクライアント端末側でAWS Encryption CLIを用いてクライアントサイド暗号化を行う。さらにKMSキーポリシーを利用し、「推論を実行するSageMakerエンドポイントのRole」や「特定のデータエンジニアリング用Lambda」にのみDecrypt権限を付与するゼロトラストな設計を構築する。
- **副業での需要**: ★★★☆☆
  - セキュリティ要件の厳しい大企業や金融・ヘルスケア系のデータ基盤構築案件において必須となるアプローチ。「ただ動くML基盤」ではなく「セキュアでコンプライアンス要件を満たしたMLOps環境」を設計・構築できるエンジニアは、単価の高いアーキテクトやコンサル案件（時給5000円〜）で高く評価される。

[Amazon SageMaker: DeepARアルゴリズムのサポート入力形式]

- **概要**: SageMaker組み込みの時系列予測アルゴリズムである「DeepAR」がサポートするトレーニング用入力データ形式は「JSON Lines」と「Apache Parquet」の二種類のみである。他の多くの組み込みアルゴリズムで標準的に使われる「RecordIO-Protobuf」形式はサポートしていない。
- **試験ポイント**:
  - 時系列分析（DeepAR）問題における「データフォーマット」の選択肢として、「RecordIO-Protobuf形式に変換してS3に配置する」という選択肢はサポート対象外のため明確な誤り（引っかけ）となる。
  - DeepARに対する正しい前処理のアプローチは、「データをJSON Lines形式（またはParquet形式）にフォーマットする」である。
  - 一方で、Linear Learner（線形学習器）やXGBoostなど他のアルゴリズムにおいては「RecordIO-Protobuf」が推奨されることも多いため、アルゴリズムごとの対応フォーマットの違いを混同しないようにする。
- **実務での使い方**: 売上予測やインフラのリソース予測などを行う際、AthenaやRedshiftからQueryした生データをSageMaker Data WranglerやAWS Glueを用いてETL処理する。この際、DeepAR向けの最終出力フォーマットとして、時系列ごとの特徴量やターゲットを含むJSON Lines形式やParquet形式に成形する処理をパイプライン（SageMaker Pipelines等）に組み込む。
- **副業での需要**: ★★☆☆☆
  - 組み込みアルゴリズムの入力フォーマットエラーは実務でも初心者が陥りやすいエラーの一つ。ただ、フォーマット変換自体はPandasなどで容易なため案件需要に直結するわけではないが、「需要予測のPoC構築（時給4000円〜）」等において、アルゴリズム選定とデータ準備の手戻りを防ぐための基礎スキルとして必須。

[Amazon SageMaker: Model Registryを用いた承認ワークフローとデプロイ管理]

- **概要**: 開発・学習したモデルを本番エンドポイントへデプロイする際、直接デプロイするのではなく、まずは「SageMaker Model Registry」に登録してバージョンや精度指標などのメタデータを管理し、適切な承認プロセスを経ることが本番運用時のベストプラクティスである。
- **試験ポイント**:
  - 本番運用（MLOps）のパイプライン構築が問われる問題において、「学習完了後に直接エンドポイントへデプロイする」という選択肢は、技術的には可能であっても品質管理の観点で不適切（引っかけ）とされることが多い。
  - 要件を満たす正解のワークフローは、「Model Registryにモデルを登録する」→「精度指標等を確認し、承認プロセス（手動/自動）を経る」→「承認（Approved）されたモデルのみを本番エンドポイントへデプロイする」となる。
- **実務での使い方**: SageMaker Pipelinesを利用した自動化ワークフロー（CI/CT）内で多用される。学習ステップの最後にモデルをRegistryへ `PendingManualApproval` ステータスで登録する。その後、MLエンジニアや管理者が評価レポートを確認してUI上からステータスを `Approved` に変更し、それをEventBridgeで検知して本番環境デプロイ用のパイプライン（AWS CodePipelineなど）を自動トリガーするアーキテクチャが定石。
- **副業での需要**: ★★★★☆
  - 企業の実証実験（PoC）フェーズから「本番運用（Production）フェーズ」へ移行する際のMLOps基盤構築案件において必須となる設計コンセプト。「推論できるスクリプトを書け

る」レベルから脱却し、「安全にモデルを更新・運用できる自動化パイプライン」を設計・構築できると、高単価（時給5000〜8000円）のインフラ・MLOps案件で重宝される。

[Amazon SageMaker: ノートブックインスタンスのライフサイクル設定 (LCC)]

- **概要**: ライフサイクル設定（Lifecycle Configuration: LCC）を使用すると、SageMakerノートブックインスタンスの「作成時（on-create）」や「起動時（on-start）」に自動でカスタムシェルスクリプトを実行し、依存パッケージのインストールや環境設定を自動化できる。
- **試験ポイント**:
  - 複数の新しいノートブックインスタンスに共通のパッケージやスクリプトを「運用オーバーヘッド（手間）を最小限に設定したい」という要件に対し、「カスタムコンテナを自作する」「AMIを別途作成する」といった選択肢は管理コストが高いため不適切（引っかけ）。正解は「ライフサイクル設定を作成し、インスタンスにアタッチする」である。
  - 追加のインフラ（EC2など）や外部サービスを導入することなく、SageMakerのネイティブ機能のみで環境の標準化が完結する点が重要。
- **実務での使い方**: データサイエンスチーム向けに標準化された分析環境を構築する際に多用される。CloudFormationやTerraformでノートブックインスタンスをプロビジョニングする際、全社共通のGitリポジトリ（初期コード）のクローン、社内プロキシ設定、特定バージョンのPythonパッケージの `pip install` 処理などを記述したLCCスクリプトを紐付け、開発環境のセットアップを完全に自動化する。
- **副業での需要**: ★★☆☆☆
  - 近年はよりモダンなSageMaker Studioへの移行が進んでいるため単体での需要は減りつつあるが、「分析基盤のIaC化・標準化」というMLプラットフォーム構築案件において知っておくべき知識。面倒な環境構築やパッケージ依存関係のエラーを省き、分析チームのオンボーディングを効率化する仕組みとして提案できる。

[Amazon SageMaker: auto-stop-idleスクリプトとLCCの実行セクション]

- **概要**: `auto-stop-idle` は、ノートブックのアクティビティを監視し、指定されたアイドル時間が経過した後に自動的にノートブックインスタンスを停止し、不要なランニングコストを削減するためのスクリプト。
- **試験ポイント**:
  - ライフサイクル設定（LCC）には、「Create Notebook（インスタンス作成時に1回だけ実行）」と「Start Notebook（起動・再起動のたびに実行）」の2つの実行タイミングが存在する。
  - アイドル停止スクリプトは継続的に監視タスクを稼働させる必要があるため、必ず「**Start Notebook（起動時）**」セクションに配置しなければならない。これを「Create Notebook セクションに配置する」としている選択肢は誤り（引っかけ）となる。
- **実務での使い方**: 企業でSageMakerを導入する際の「シャットダウン忘れによる過剰課金」を防ぐための標準的なコスト最適化プラクティス。GitHub（AWS公式サンプル）で公開されている `auto-stop-idle` スクリプトをベースに、自社の要件（アイドル判定を60分にする等）に合わせてパラメータを調整し、IaC（Terraformなど）で新しいノートブックの「Start Notebook」にデフォルトで紐付ける。
- **副業での需要**: ★★★☆☆

- MLプラットフォームや分析環境のインフラ構築案件において、「コスト最適化（FinOps）」の観点から自発的に提案・実装すべき仕組み。提供されているスクリプトを適切にアタッチするだけで稼働料金を劇的に抑えることができるため、クライアントからの信頼を得やすく、継続的な保守案件に繋がりやすい。